Break work into AI-executable tasks that deliver with Github Copilot

Description

Artificial Intelligence is rapidly changing how software is designed, developed, tested, and maintained. Copilot is one of the leading AI-first development environments, combining the familiarity of a modern IDE with powerful AI capabilities that can accelerate development and transform the way developers work.

In this hands-on introductory lab, you'll explore the fundamentals of Copilot and learn how AI-assisted development fits into the modern software engineering workflow. Through guided exercises, you'll discover how to use natural language to generate code, understand existing applications, create tests, refactor implementations, and navigate large codebases more effectively.

This lab focuses on taking a broad, real-world feature ticket and systematically breaking it down into small, dependency-ordered task specifications using the Inputs-Actions-Outputs-Constraints framework. Working directly inside Copilot , delegates will sequentially execute their planned specs to build out a complete feature layer by layer. Through this practical exercise, participants master how precise task scoping, explicit file referencing, and logical execution sequencing prevent AI context pollution and eliminate code rework.

By the end of this session, delegates will be able to break ambiguous feature requirements into structured, dependency-aware task specifications that Copilot AI can execute accurately without breaking existing code.

Learning Outcomes

By completing this lab, you will be able to:

- Spot hidden assumptions in feature tickets so you can catch vague requirements before Copilot guesses wrong and breaks your code.
- Use the Inputs-Actions-Outputs-Constraints framework to write prompts that tell Copilot exactly what files to read, what code to produce, and what rules to follow.
- Break big feature tickets into small, single-focused tasks that touch only 1 or 2 files at a time to prevent AI context overload.
- Map out the correct sequence of work so foundational code—like types and schemas—is built before dependent tasks run.
- Structure tasks to prevent execution loops and eliminate constant jumping between different layers of your tech stack.
- Steer Copilot using native @file tags and rules to feed precise codebase context into your prompts without typing long explanations.
- Pass clean outputs from one task into the next while managing chat context so Copilot never rewrites earlier working code.

Target Audience

This lab is designed for technical leads, delivery managers and technology professionals who want to understand how AI-powered development tools are changing the software delivery lifecycle and how to get started with Copilot.

Task outline

Given this Business Scenario:

Your security team requires all incoming requests to /api/* endpoints to be logged to a local JSON file (audit.log). However, incoming payloads often contain sensitive fields like passwords, credit card numbers, or tokens that must be redacted (replaced with "***REDACTED***") before writing to disk.

Below, we have broken down this single requirement into 3 manageable ones so that Copilot is given 3 distinct tasks.

Before writing any prompts, open up a Notepad file, and list the three tasks to complete in this exercise. This will help you to plan out your prompts for each task below using the IOC format (Input-Output-Constraint):

Task 1 Spec: Define data interfaces (AuditEntry, RedactionConfig) in a separate types file.
Task 2 Spec: Create utility functions (redactPayload, appendAuditLog) in a services file.
Task 3 Spec: Build the Express middleware and attach it to server.ts.

Try all tasks first, but if you are struggling refer to: https://github.com/jordanbenbelaid/break-work-into-sol/blob/main/README.md for sample solutions

Task 1:

1. Open Copilot (From within VSCode, in the folder you want all following files to be saved in).
2. Reference @src/types/index.ts in your prompt so Copilot understands existing project conventions.
3. Paste your Task 1 Spec. Be sure to include explicit Constraints (e.g., "Do NOT modify route files, do NOT install external packages").
4. Review the generated code diff. If Copilot created @src/types/audit.ts cleanly, hit Accept.
5. Run npm run typecheck in your terminal to verify zero build errors.

Task 2:

1. Open a new, clean prompt session in Copilot .
2. Reference your newly created type file using @src/types/audit.ts.
3. Paste your Task 2 Spec asking for (you will need to have these functions created by Copilot):
   redactPayload(): A pure function that replaces sensitive keys (password, token, secret) with "***REDACTED***".
   appendAuditLog(): A file-writing function using Node's native fs.appendFileSync.

4. Guardrail check: Ensure Copilot didn't invent third-party npm logging packages (like Winston or Pino). If it did, reject the change and re-add the constraint: "Use Node native fs only."
5. Accept the code diff and verfiy it worked with "npm run typecheck"

Task 3:

1. Reset your prompt session.
2. Reference both prerequisite files: @src/types/audit.ts and @src/services/auditService.ts.
3. Paste your Task 3 Spec to create the Express middleware wrapper.
4. Explicitly direct Copilot to intercept the response completion (res.on('finish')) so it captures the final HTTP status code.
5. Have Copilot mount the new middleware in @src/server.ts before the /api routes.

Verify:

Start your dev server in the temrinal: npm run dev

Open a seperate terminal and send a test request with a sensitive payload (Use this prompt exactly or feel free to experiment):

```
curl -X POST http://localhost:3000/api/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "password": "SuperSecretPassword123"}'
```

Open audit.log in your file tree and verify that the password was successfully replaced with "***REDACTED***".